



Argus

Complete Setup & Usage Guide — every page, every feature, every integration,
from first install to a multi-site deployment.

Built by ZTPL · ztplsolutions.com
argus.ztplsolutions.com

Argus — Full Guide

Everything about the project in one document: how every page works, what runs behind the scenes, how network reachability actually works for multi-site deployments, security posture, how to build and release it, how to host the download site and issue licenses, a client setup walkthrough, a pre-sale verification checklist, troubleshooting, and the full CLI/env-var/file reference. Read top to bottom once; use the table of contents to jump back in after that. See [README.md](#) for the short version.

Contents

1. [What Argus is](#)
 2. [Architecture at a glance](#)
 3. [What runs in the background](#)
 4. [Every page — what it does, how to use it](#)
 5. [Network reachability & multi-site / VPN deployments](#)
 - [5.1 Remote agents for segmented networks without a VPN](#)
 6. [Security posture](#)
 7. [Building from source](#)
 8. [Cutting a release](#)
 9. [Hosting the download page & selling to clients](#)
 - [9.1 Host the download page](#)
 - [9.2 The manual-sale workflow, start to finish](#)
 - [9.3 License plans](#)
 - [9.4 Issuing a license from a UI instead of the CLI](#)
 - [9.5 Issuing a license from anywhere \(hosted portal at /admin\)](#)
 - [9.6 Website deploy details \(Vercel, contact-form email\)](#)
 - [9.7 Setting up the escalation hierarchy \(Tier 1 / 2 / 3 contacts\)](#)
 10. [Client setup walkthrough](#)
 11. [Verification checklist](#)
 12. [Troubleshooting](#)
 13. [Reference: CLI flags, environment variables, file locations](#)
-
-

1. What Argus is

Argus discovers devices on a network, monitors them continuously (ICMP, TCP, HTTP/HTTPS, SNMP v1/v2c/v3, and — for FortiGate and Sophos firewalls specifically — the vendor's own management API), raises alerts the moment something goes down or degrades, and notifies the right people by email, webhook, Slack, Microsoft Teams, or PagerDuty, walking a tiered escalation chain (Tier 1 → Tier 2 → Tier 3...) if nobody acknowledges in time. It ships as a Windows installer — run the wizard once and Argus installs itself as a background Windows service: no console window, no need to keep anything open,

and it keeps running across logout/reboot. Everything it stores (device inventory, metrics history, alerts, settings) lives in a local SQLite database next to the exe. No account, no cloud dependency, no internet connection required to run it.

The business model this is built for: you sell a copy directly, collect payment however you choose (bank transfer, UPI, invoice — outside the app entirely), then issue a signed license file for one of the named device-range plans (see [§9.3](#)) and email the customer both that and the installer. No self-service signup, no recurring billing inside the product.

2. Architecture at a glance

Hexagonal architecture — dependencies point inward, and each layer only knows about the one beneath it:

domain/	Pure business logic. No I/O, no imports from anywhere else. State machines, classification rules, the SSRF guard, license/update-feed signature verification.
application/	Use cases — orchestrates domain logic against ports/ interfaces. Never imports a concrete adapter directly.
ports/	TypeScript interfaces only — the contracts application/ depends on (repos, notifiers, clock, queue, license service, etc).
adapters/	Concrete implementations of those ports — SQLite/Postgres repos, SMTP/webhook notifiers, the network checkers, the Ed25519 license/update-feed verifiers.
api/	HTTP routes (Hono) — thin: validate input, call an application use case, serialize the response. Auth/CSRF/rate-limit middleware lives here.
bootstrap/	The composition root — wires concrete adapters into the container, CLI arg parsing, Windows service/tray/firewall/self-update helpers, process entry points.

An ESLint rule enforces the dependency direction (`domain` / `application` cannot import from `adapters` / `api` / `bootstrap`) — it's not just a convention, a violation fails `bun run lint`.

Single-tenant, self-hosted exe — one customer, one SQLite file, no accounts to manage, no billing integration (manual license files instead). There is no multi-tenant/hosted mode; every repo method still takes a `tenantId` for internal consistency, but the exe only ever has one row (`"local"`).

Stack: Bun + TypeScript on the backend (Hono for routing, `bun:sqlite` for the database, no Node.js runtime needed), React 18 + Vite + Tailwind + Recharts on the frontend, compiled together into one native Windows binary via `bun build -compile`.

3. What runs in the background

The moment the exe starts, five things spin up alongside the HTTP server:

- **Scheduler** — polls every enabled device/check on its configured interval, respecting a concurrency cap (default 50 in flight at once). Load-tested to 10,000 devices.
- **AlertEngine** — a state machine per device (`up` → `degraded` → `down` → `flapping` → `maintenance`). Deduplicates repeated failures into one open alert, aggregates a "storm" of many devices failing at once into a single notification instead of flooding you, and auto-resolves + notifies on recovery.
- **EscalationWorker** — walks a device group's escalation chain (e.g. notify the owner at 0min, Tier 1 at 10min, Tier 2 at 30min, Tier 3 at 60min) and stops the moment someone acknowledges. Set this up in Inventory → Groups → "Edit escalation" — see [§9.7](#).
- **RetentionScheduler** — rolls raw metrics up into hourly aggregates and prunes old data on the schedule set in Settings → General (default: 30 days raw, 365 days rolled up).

- **WebSocket broadcast** — pushes device-status and alert changes to every connected browser tab live, which is why the Dashboard updates without a manual refresh.
- **HeartbeatScheduler** — pings an external dead-man's-switch URL (healthchecks.io, Cronitor, Uptime Kuma, or similar) every 5 minutes if one's configured (Settings → General → Heartbeat URL). This is the one failure mode nothing else in Argus can detect from the inside: if the whole process, the Windows service, or the machine itself dies, only something watching from outside can notice a run of missed pings and page someone.
- **ReclassificationScheduler** — once shortly after startup, then once a day, re-probes SNMP sysDescr for every device still sitting at type "unknown" with SNMP configured, and reclassifies it if a confident ($\geq 80\%$) signal now exists. Never touches a device that already has a specific type, whether the classifier set it originally or an admin corrected it by hand.

Two more things happen on-demand, not continuously:

- **License enforcement** — every device-create call (single or bulk import) checks the current license state and refuses once you're at the device cap. Trial mode (no license applied) caps at 5 devices.
- **Self-update** — only when you click "Check for updates" / "Update now" in Settings → General. Downloads the new exe, verifies it's authentically signed by you (see §6), swaps it into place, and relaunches — all in about 10 seconds.

4. Every page — what it does, how to use it

All routes below are under the app once it's running (e.g. <http://localhost:58070/inventory>).

Login (</login>)

Email + password. The first account is created via Setup, below; there's no public signup.

Setup (</setup>) — first run only

Shown automatically the very first time the app starts and no admin account exists yet. Set an instance name, create the admin email/password, and accept the terms of use. After this, </setup> redirects to </login> — it can't be run twice.

Dashboard (</>)

The home screen. Top strip: total devices, up/degraded/down counts, open alerts, availability today. Below that: fleet-health and alert-severity composition bars, an availability meter, device mix by type, latency leaders (which devices are slowest to respond), and devices-by-group. The main panel is a searchable, filterable device grid (or table — toggle in the top right) — click any device to open its detail page. The side panel is a live alert feed.

Inventory (</inventory>)

The full device list as a table, with add/edit/delete. Click "+ Add device" to add one manually (name, IP, type, group, poll interval, whether it answers HTTP/HTTPS, optional SNMP credentials). Click a row to edit it. The IP address can't be changed after creation — delete and re-add if it changes. Sortable by state (down-first by default), with a critical-asset (⚡) indicator next to flagged devices. Groups (used for escalation chains and dashboard breakdowns) are managed at the bottom of this page — click "Edit escalation" on a group to set up its Tier 1/2/3 alert contacts and the delay before each tier fires (see §9.7).

SNMP (edit device → SNMP section): pick v1, v2c (community string), or v3 (username + security level + auth/priv protocols and keys). Same credential UI regardless of device type — a switch, a router, or a firewall from any vendor all use it identically.

Vendor API (edit device → Vendor API section, optional, on top of any SNMP/HTTP checks above) — for a FortiGate or Sophos firewall specifically:

- **FortiGate**: paste a REST API admin token. Polls CPU, memory, active sessions, IPsec VPN tunnel status, and identity facts (model, firmware, serial, HA role) straight from the device.

- **Sophos** (standalone on-prem SFOS/XG): enter a dedicated API username + password (port defaults to 4444, not 443 — Sophos reserves 443 for the SSL VPN portal on most models). Validates the device is reachable and reports IPsec VPN tunnel status. Sophos's classic on-prem API has no live CPU/memory endpoint the way FortiGate's does — pair with SNMP above for those two metrics.

Either way, check the **Firewalls** page (below) for the fleet-wide view once configured.

Remote agent (edit device → Remote agent dropdown, optional) — assigns this device to a remote poller agent instead of being checked locally. See §5.1.

Discovery ([/discovery](#))

Enter a subnet in CIDR form (e.g. [192.168.1.0/24](#) , capped at [/22](#) — 1024 addresses — to protect the host doing the scanning) and click "Start scan." Argus pings every address, probes common ports, tries SNMP, and guesses each device's type from what it finds. Review the results, adjust types if needed, select which ones to import, and click "Add N devices." Already-known IPs are marked and pre-deselected. For a multi-subnet estate, run one scan per subnet — see §5.

Device Detail ([/devices/:id](#))

Reached by clicking any device elsewhere in the app. Four tabs:

- **Metrics** — latency, packet loss, and CPU/memory charts (from SNMP or, for a FortiGate, the vendor API) over a selectable time range (1h/6h/24h/7d/30d). A FortiGate or Sophos device with a vendor API configured also gets Active Sessions (FortiGate only) and VPN Tunnels Up (both) charts.
- **Alerts** — every alert this device has ever raised.
- **Checks** — the ICMP/TCP/HTTP/SNMP/vendor-API checks running against it; toggle them on/off and edit latency/loss alert thresholds inline. If a check is currently failing, its last error message (e.g. "system/status HTTP 403", "The operation timed out") shows right there — this is the check's own health, independent of the device's overall up/down state: ICMP is authoritative for reachability, so a misconfigured secondary check never marks a genuinely reachable device down, but its own failure is still visible here for diagnosing.
- **Maintenance** — schedule a maintenance window (suppresses alerting) or cancel one.

Map ([/map](#))

A force-directed topology diagram — devices connected to their group, groups connected to a core node. Click a node for its details in the side panel. Drag a node to reposition it (saved automatically). Drag the background to pan, scroll to zoom, or use the +/-/reset controls in the corner.

Bandwidth ([/bandwidth](#))

SNMP interface throughput (in/out) for devices with SNMP configured — view aggregate and per-interface charts over 1h/6h/24h. Click **Discover** next to a device to list every interface it actually reports (name, index, up/down) over a live SNMP walk, and tick which ones to track — no more hand-snmpwalking a device outside Argus to find index numbers first. Manual index entry still works too, for anyone who'd rather just type them.

Firewalls ([/firewalls](#))

Fleet-wide view of every device with type "Firewall" — live CPU/memory/session/VPN-tunnel stat tiles per device (from SNMP and/or a configured vendor API), plus model/firmware/HA-role badges for anything a FortiGate or Sophos vendor API has reported. A device with neither SNMP nor a vendor API configured shows a "no SNMP/API creds configured" hint instead of blank tiles.

Alerts ([/alerts](#))

Every alert, filterable by status (open/acknowledged/resolved) and severity (critical/warning/info). Acknowledge silences escalation for that alert; Resolve closes it (also happens automatically when the device recovers).

SLA ([/sla](#))

Availability chart and table per device against a target (default 99.9%), plus three stat tiles: devices meeting target, total downtime, and the worst performer. Filter by group and time period.

Reports ([/reports](#))

A day-by-severity alert-volume heatmap and a scheduled email report (sends the SLA summary above plus an open-alerts summary, on whatever recurrence you set). Filter by group and time period (24h/7d/30d).

Notifications ([/settings/notifications](#))

Two parts on this page. The top cards (SMTP, Webhook, Syslog) are **instance-wide** — admin-only, configure once for the whole install: mail server credentials for email alerts, a webhook URL + optional HMAC signing secret, and/or a syslog (CEF) forwarding target for an existing SIEM. Each has a "send test" button. The bottom card is **per-user**: which channels you personally want alerts on, your minimum severity, and quiet hours. Six channels, each with its own target:

- **Email** — to your account's address, via the instance SMTP above.
- **Webhook** — your own URL, separate from the shared instance-wide one above.
- **Slack** — a Slack Incoming Webhook URL (create one under your workspace's Incoming Webhooks app). Posts a severity-colored message.
- **Microsoft Teams** — a channel's Incoming Webhook connector URL. Posts a card with a one-click Acknowledge action.
- **PagerDuty** — a service's Events API v2 integration key (not an account-wide API token). Resolving the alert in Argus resolves the PagerDuty incident too, not just a second notification.
- **Syslog** — instance-wide only, no per-user target (it's a fire-and-forget forward, not a personal notification).

Plus an opt-in personal digest email (daily/weekly SLA + open-alerts summary), independent of the real-time channels above.

Security ([/settings/security](#))

Change your own password, and set up two-factor authentication (TOTP — any standard authenticator app).

Admin → Users ([/admin/users](#))

Invite teammates (email, role, temporary password — they're forced to change it on first login), change roles (viewer/operator/admin), disable/enable accounts, and view/revoke a user's active sessions.

Admin → Audit Log ([/admin/audit](#))

Every sensitive action (logins, device changes, settings changes, backup/restore, license applications) with who/when/what, filterable by action — shown as a timeline, newest first.

Admin → General Settings ([/admin/settings](#))

Instance name, port, log level, default poll interval and concurrency, retention windows, the auto-update feed URL, and backup/restore (download a full [.zip](#) of the database + config, or restore from one — the app restarts automatically after a restore).

Admin → API Keys ([/admin/api-keys](#))

Create/revoke keys for the public REST API ([/api/v1/...](#) , documented at [/api/docs](#) when logged in as admin) — for scripting or integrating with something else you run. Enforced read-only at the route level, not just by convention.

Admin → Remote Agents ([/admin/remote-agents](#))

Create/revoke remote poller agents for network segments this Argus instance can't reach directly. Click "Create agent," name it, and copy the one-time token shown (it's never shown again). See §5.1 for running the agent itself and assigning devices to it.

Admin → License (/admin/license)

Shows current license status (trial / licensed / renewal overdue / expired / invalid), customer name, plan, device usage against the cap, and expiry. Paste a `.license.key` file's contents here and click "Apply license" — see §9 for how you issue one after a sale.

Admin → SSO / SAML (/admin/sso)

Configure a SAML 2.0 identity provider (Okta, Azure AD/Entra ID, Google Workspace, etc.) so users can sign in through your existing IdP instead of an Argus-local password.

Admin → Status Page (/admin/status-page)

Configure the public, unauthenticated status page (below) — which groups/devices are visible, the page title, and whether it's published at all.

Public Status Page (/status/:tenantSlug)

Unauthenticated — anyone with the link can see it. Shows an overall operational/degraded/outage banner, per-group status, and recent incident history. Only device names and up/down/degraded state are ever shown — no IPs, config, or credentials.

5. Network reachability & multi-site / VPN deployments

By default, Argus is **agentless**: every check (ICMP ping, TCP connect, HTTP/HTTPS, SNMP, a vendor API) is a normal outbound network call made **from the single machine running Argus.exe**. For most multi-site setups the real question is never "does Argus support multi-site" — it's **"does this one machine have a network path to the other sites,"** exactly as if you opened a command prompt on it and typed `ping 10.20.30.5`. If there's a working site-to-site VPN, one instance genuinely can monitor every site through it — see the worked example below.

For a segment with **no VPN at all** — no routed path back to wherever Argus runs, full stop — a lightweight **remote agent** can run inside that segment instead and push results back over HTTPS. See §5.1.

Worked example

Four offices, each on its own subnet, all site-to-site VPN'd through one hub location's firewall. Argus runs on a PC at the hub.

Yes — that one PC can monitor devices at all four sites, provided:

1. **The VPN is actually site-to-site with routing, not remote-access-only.** The hub firewall must have a route to each remote subnet over its tunnel, and the far-end firewall must route back. This is VPN/firewall configuration — entirely outside Argus, and the single most common reason a "multi-site VPN" doesn't reach where you expect.
2. **No two sites use the same subnet.** If Site A and Site C both hand out `192.168.1.0/24` (extremely common with factory-default routers, since nobody planned for a merger), the VPN literally cannot route to both without NAT remapping one of them first. Check this before anything else — it's the #1 real-world blocker for exactly this scenario.
3. **Firewall rules between the hub and each site allow the specific traffic Argus sends:**
 - **ICMP** (ping) — many firewalls block ICMP across a tunnel by default; if blocked, ping-based checks report a perfectly healthy device as down.
 - **The exact TCP/HTTP(S) port** each configured check targets.

- **UDP 161 (SNMP)**, only if you want CPU/memory metrics from that device.
4. **The hub PC's own local firewall** (Windows Defender Firewall on the box itself) isn't blocking outbound probes — `Argus.exe --fix-firewall` prints the exact commands for this one, but it only covers the local machine, not the network beyond it.
 5. **You add the device by its real IP** — Argus has no concept of "site" as a network construct. **Groups** (Inventory → Groups) are a manual label you create yourself, purely for organizing escalation chains and dashboard breakdowns. Nothing stops you from making one Group per office and assigning devices to match — that's organizational, and doesn't affect reachability at all.
 6. **Discovery scans one subnet at a time**, capped at `/22` (1024 addresses) per scan. For four sites on four subnets, run Discovery once per subnet from the same instance (e.g. `192.168.1.0/24`, then `192.168.2.0/24`, ...) — each scan only finds what's actually reachable from the hub over the tunnel at that moment.

A real limitation, not a bug

This architecture cannot distinguish "the device itself is down" from "the VPN tunnel to that whole site is down" — if the tunnel to Site B drops, every device at Site B reports down at once. That's *correct* from the hub's point of view (nothing there is reachable right now).

This does **not** mean you lose track of which devices are actually down, though: every individual device alert is still created and shown by name in Inventory, Dashboard, and Alerts, and Argus only folds the *notifications* (email/webhook) into a single message once 20+ devices go down within the same 60-second window — purely so you get one email instead of twenty, never so you lose the detail. That combined notification explicitly lists every affected device by name and IP (both in the email body and as a structured `affectedDevices` array in the webhook JSON, for anything downstream parsing it), so the on-call person still knows exactly what to go check.

What you genuinely can't get from a single hub instance is *why* — whether it's 20 unrelated hardware failures or one shared upstream link (the VPN tunnel itself) taking a whole site down at once. If a storm's device list all belongs to one office/group, that's a strong hint it's the link, not the devices — but confirming that for certain (and getting a monitoring point that keeps working even while that tunnel is down) needs a second, independent Argus instance physically inside that site, rather than relying on the hub to see through a dead tunnel.

Two more things worth planning for

- **Latency thresholds** — a check made over a VPN hop is slower than one made on-LAN. Don't reuse an on-LAN latency threshold (e.g. 20ms) for a device that's normally 40–60ms away over WAN — set realistic per-check thresholds (Device Detail → Checks) or it'll flag "degraded" constantly.
- **Scale** — default concurrency (50 simultaneous checks) and the load-tested ceiling (10,000 devices) comfortably cover four sites' worth of devices; this is not a scaling concern at that size.

Bottom line: one Argus instance genuinely can monitor multiple sites through a single VPN hub — Argus adds no prerequisite beyond "can this machine reach that IP, on that port." Everything that determines whether a specific remote device gets monitored is routing and firewall configuration on the network itself, not anything inside the product.

5.1 Remote agents for segmented networks without a VPN

When there's no site-to-site VPN at all — a segment genuinely unreachable from wherever the central Argus instance runs — a small standalone agent process runs inside that segment instead, checks its assigned devices locally, and pushes results back to the central instance over HTTPS. It's a separate, deliberately minimal process (`bun run agent`, or `bun run src/bootstrap/agentMain.ts` if building from source) — not a mode flag on `Argus.exe`, not installed by the Windows installer.

Set it up:

1. **Central instance:** Admin → Remote Agents → "Create agent" → name it (e.g. `branch-office`). Copy the token shown — it's a one-time display, same as an API key.
2. **On a machine inside the segment** (any OS Bun runs on — doesn't have to be Windows, doesn't need the full Argus install): set two environment variables and run the agent:


```
ARGUS_AGENT_TOKEN=argus_agent_... \
ARGUS_CENTRAL_URL=https://central.example.com \
bun run agent
```

Optional: `ARGUS_AGENT_INTERVAL_MS` (default `30000`) to change how often it polls its assigned devices. Keep it running as a scheduled task, a systemd unit, `pm2`, or whatever process supervisor fits that machine — there's no built-in service registration for the agent the way the installer does for the main product.

3. **Central instance:** Inventory → edit each device in that segment → "Remote agent" dropdown → pick the agent you created. That device is now checked by the agent instead of locally.

What an agent can and can't do: ICMP, TCP, and HTTP/HTTPS checks only — the same three that need no credential material. SNMP and vendor-API checks (which need encrypted credentials the agent has no secure way to hold) still run centrally, so a device split across both needs to be reachable from the central instance for those specifically even while its ICMP/TCP/HTTP checks run from the agent. Results flow through the exact same state-machine, alerting, and metrics pipeline a locally-pollled device uses — an agent-monitored device looks identical everywhere in the UI to a local one, except for which process actually reached it.

6. Security posture

This codebase went through a full security audit (two independent expert passes covering authentication, session/CSRF handling, cryptography, SQL query safety, SSRF, file/path handling, and tenant isolation) and every confirmed finding was fixed and verified against the real running app, not just reasoned about:

- **Password hashing:** argon2id. **Secrets at rest** (SNMP credentials): AES-256-GCM with a fresh random IV per value. **API keys:** bcrypt. **Session tokens, CSRF tokens, ack-links:** 256-bit random, compared with constant-time comparison (not vulnerable to timing attacks).
- **Rate limiting** (login, signup, setup) is keyed off the real TCP connection address, not a client-suppliable header — it can't be bypassed by sending a different `X-Forwarded-For` on every request.
- **The auto-update mechanism requires a cryptographic signature**, not just a checksum. A checksum alone doesn't prove a downloaded binary is authentic if it's sourced from the same channel as the download — the update feed must be signed with an Ed25519 key that never leaves your machine (deliberately a *different* key than the one that signs customer licenses), and both the feed URL and the download URL must be `https://`. See §8.
- **Webhook notification targets** are blocked from pointing at loopback, link-local (including the cloud metadata endpoint `169.254.169.254`), and private/internal network ranges — a low-privilege user configuring their own alert webhook (`PUT /notification-prefs` only requires `viewer`) can't turn it into an internal network scanner. This guard applies only to webhook destinations, never to monitored device IPs — private/internal addresses are exactly what an NMS is for. Covers both IPv4 and IPv6 (loopback `::1`, link-local `fe80::/10`, unique-local `fc00::/7`, and IPv4-mapped `::ffff:a.b.c.d` addresses unwrapped to their embedded IPv4) — an earlier version only checked IPv4 literals, so a hostname resolving to just an IPv6 address skipped validation entirely.
- **Session cookies** are `httpOnly` and `SameSite=Lax` always. They are **not** marked `Secure`: Argus has no built-in TLS termination and is designed to be reachable over plain `http://` on a LAN (that's the whole point of the auto-added firewall rule — a colleague hitting `http://<this-pc>:58070` from another machine on the same network); a `Secure` cookie would simply never be sent back over that connection, breaking login entirely. **If you expose Argus beyond a trusted LAN** (e.g. port-forwarded to the public internet), the session cookie travels in cleartext on that path — put a TLS-terminating reverse proxy (nginx, Caddy, IIS ARR) in front of it first. Don't port-forward 58070 directly to the internet.
- Every database query across both the SQLite and Postgres adapters is parameterized — no SQL injection surface anywhere in the codebase.
- Backup restore only ever extracts two fixed, hardcoded entry names from the uploaded archive (never attacker-controlled paths) — no zip-slip / path-traversal surface.

- Security headers (`X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , a strict `Content-Security-Policy` with no `script-src 'unsafe-inline'`) are set on every response, and no `Access-Control-Allow-Origin` header is ever sent — cross-origin requests are refused by default.
- Dependencies are audited (`bun audit`) before every release; anything fixable within a compatible version range is applied. Remaining flagged advisories at time of writing are confined to build-time-only tooling (ESLint's dependency tree, Vite's dev server, the Vercel deploy SDK) — nothing in that list ships inside the compiled exe or runs in a customer's browser.

No system is ever "permanently secure" — this reflects the state of a real, tools-assisted audit at the time of writing, not a guarantee. Re-run a review after any significant change, especially to auth, crypto, or anything that fetches/executes external input.

7. Building from source

Requires `Bun` (no Node.js needed).

```
bun install          # backend deps
cd ui && bun install && cd .. # frontend deps
bun run dev          # dev server: backend on :58070 (watch mode), Vite on its own port
```

```
bun run test          # full test suite
bun run lint          # ESLint, including the hexagonal-boundary rule
bunx tsc --noEmit     # backend typecheck
cd ui && bunx tsc --noEmit # frontend typecheck
```

8. Cutting a release

One-time setup (only ever done once, or after a deliberate key rotation):

```
bun run scripts/generate-license-keypair.ts # -> secrets/license-private-key.pem
bun run scripts/generate-release-keypair.ts # -> secrets/release-private-key.pem
```

Both private keys land in `secrets/` (gitignored — never commit them, never lose them). Back them up somewhere durable (password manager, encrypted archive). Losing the license key means you can't issue new licenses (old ones still verify fine); losing the release key means you can't sign new auto-updates.

Also one-time: install `Inno Setup` (free) so `bun run release` can build the wizard installer, not just the portable exe:

```
winget install JRSoftware.InnoSetup
```

If `ISCC.exe` ends up somewhere `bun run release` doesn't check by default, point it there with the `ISCC_PATH` environment variable. Without Inno Setup installed at all, the release step that builds the installer just prints a warning and skips itself — you still get the portable exe.

Optional, and also one-time: to have the installer register Argus as a Windows service automatically (rather than the customer's machine printing manual setup instructions the first time `--install-service` runs), download `WinSW` once (`WinSW-x64.exe`) and place it at `tools/Argus-service.exe` (gitignored, kept out of source control deliberately — it's a third-party binary, not something to vendor into the repo). The installer script picks it up automatically if present, and silently skips bundling it if not.

Optional: set `COMPANY_NAME` when releasing to stamp your own business name onto the exe's version info and the installer's publisher field (defaults to "Argus" if unset):

```
COMPANY_NAME="Acme IT Services" bun run release
```

Every release:

```
bun run release
```

This one command cleans, runs the full test suite, builds the UI, compiles the exe, stamps the icon/version/company info onto it, smoke-tests the compiled binary in a clean temp directory (spins it up, hits `/api/health`, completes setup, creates a device via the real API, confirms it persisted, kills it), writes a checksum, builds the wizard installer (if Inno Setup is installed), and publishes unversioned "stable-name" aliases of both artifacts. If any step fails, nothing broken reaches `dist/`. Output:

- `dist/Argus-Setup-vX.Y.Z-win-x64.exe` (+ `.sha256`) — the installer most customers should download.
- `dist/Argus-vX.Y.Z-win-x64.exe` (+ `.sha256`) — the portable exe, for anyone who explicitly wants to run it without an installer or background service.
- `dist/Argus-Setup-win-x64.exe` / `dist/Argus-win-x64.exe` (+ `.sha256` each) — identical bytes to the two above, under a fixed filename the website's download link points at (see §9.1).

Re-running `bun run release` produces a byte-different installer each time (Inno Setup embeds a build timestamp in the wrapper), even with identical inputs — the portable exe, by contrast, is reproducible. This is harmless as long as you treat one `bun run release` invocation as "the" build for a version: don't rebuild after you've already uploaded/checksummed one, and don't publish one attempt's checksum next to a different attempt's file.

To enable auto-update for that release — after you've uploaded the installer/exe somewhere and know its final public URL (signing has to happen after the file is reachable there, since the signature covers that URL):

```
bun run scripts/sign-release.ts --version 1.2.0 \
  --url https://github.com/you/argus/releases/download/v1.2.0/Argus-v1.2.0-win-x64.exe \
  --exe dist/Argus-v1.2.0-win-x64.exe \
  --notes "What changed in this release"
```

This writes `dist/update-feed.json`. Upload its contents wherever your customers' "Update check URL" setting (Settings → General) points — that's the one file that needs to live somewhere publicly reachable over HTTPS. Customers who have that URL configured will see "Update available" next time they check, and clicking "Update now" downloads, verifies the signature, verifies the checksum, and applies it automatically.

9. Hosting the download page & selling to clients

9.1 Host the download page

There is no separate download-page project — `website/` (the marketing site) is the download page. Its "Download for Windows" button (Hero/Navbar/Footer, wired through `RELEASE.downloadUrl` in `website/src/config.ts`) links at a **same-origin static file**, `/downloads/Argus-Setup-win-x64.exe` — served straight from `website/public/downloads/` by Vercel's CDN. Clicking it downloads the file immediately: no redirect, no external host, no login wall, and nothing that can 404 because some other repo's release wasn't published.

That fixed filename stays correct release after release because `bun run release`'s last step ("Publish stable-name aliases") copies each versioned artifact to a second, unversioned filename — `Argus-Setup-win-x64.exe` / `Argus-win-x64.exe` — **directly into** `website/public/downloads/` alongside the version-pinned `dist/` copies. Commit those two files (or push the

branch Vercel deploys) and the next deploy serves the new build at the exact same URL — the site's `<a href>` never changes.

- Optionally, still upload the versioned artifacts to a GitHub Releases page (public repo, e.g. `https://github.com/<you>/argus-releases`) purely as a changelog/archive for anyone who wants a specific old version — but the live site's Download button does **not** depend on that repo or release existing. If you go this route, keep the releases repo public: a private repo forces anyone downloading, even an anonymous customer with a direct link, to log into GitHub first.

```
gh repo create <you>/argus-releases --public
gh release create v1.2.0 \
  dist/Argus-Setup-v1.2.0-win-x64.exe dist/Argus-Setup-v1.2.0-win-x64.exe.sha256 \
  dist/Argus-v1.2.0-win-x64.exe dist/Argus-v1.2.0-win-x64.exe.sha256 \
  --repo <you>/argus-releases --title "Argus v1.2.0"
```

Never flip your actual *source* repo to public just to fix downloads — that publishes your entire codebase, pricing logic, and everything else, and there's no undoing it once it's been cloned or indexed.

9.2 The manual-sale workflow, start to finish

1. Customer pays you however you collect payment — bank transfer, UPI, an invoice — entirely outside the app. Nothing in Argus processes payments.
2. On your own machine, issue a license for the plan they bought (see §9.3 for the plans, §9.4 for a form-based way to do this instead of the command line):

```
bun run scripts/generate-license.ts --customer "Acme Corp" --plan business --devices 150 --expires 2027-07-1
```

Use `--perpetual` instead of `--expires` for a one-time-purchase sale (sets the expiry 50 years out — same mechanism, just a very long renewal date). Full flag reference is in the script's own header comment.

3. This writes a `<customer-name>.license.key` file. Email it to the customer along with a link to the download page (or the installer directly).
4. The customer downloads and runs the installer, completes Setup, and pastes the license key into Settings → License → Apply license (see §10).
5. For a subscription-style sale, repeat step 2 with a new `--expires` date when they renew and send the new key the same way — applying a new license simply replaces the old one.

Without a license applied, the app runs in trial mode (5-device cap, no expiry) — a reasonable default for prospects trying it before buying, and it's also exactly what an unpaid/lapsed customer falls back to after a 14-day grace period past expiry.

9.3 License plans

Licenses are sold as named, device-range tiers (`src/domain/license.ts` , `PLAN_DEVICE_RANGES`) — `--devices` still drives what's actually enforced, the plan name is a label plus a range check that catches "sold Starter, typed 500 devices" at issue time:

Plan	Device range
Starter	1 – 25
Professional	26 – 100
Business	101 – 500
Enterprise	501 – 2000
Unlimited	2001+

Renaming a tier or moving its range later doesn't touch any already-issued license — the signed file carries its own `deviceLimit` and `plan` string, verified independently of what these ranges currently say.

9.4 Issuing a license from a UI instead of the CLI

`scripts/generate-license.ts` is the CLI. If you'd rather fill in a form:

```
bun run license-admin
```

This opens `http://localhost:4790` in your browser — customer name, plan (dropdown, with each tier's device range shown), device limit, and either an expiry date or a perpetual checkbox. It issues the exact same signed license file as the CLI (they share `scripts/lib/issueLicense.ts`), saves it under `issued-licenses/` (gitignored) and lists every license you've issued before, with a download link for each.

This is **not** part of the product you ship to customers — it's a separate script, never imported by `src/bootstrap/main.ts`, never compiled into Argus.exe, and binds to `127.0.0.1` only. Run it on your own machine after a sale closes, the same way you'd run the CLI script.

9.5 Issuing a license from anywhere (hosted portal at /admin)

Both `scripts/generate-license.ts` and `scripts/license-admin.ts` above only work from your own machine. `website/` also ships an `/admin` page — same signing logic (`website/lib/license.ts` , a self-contained copy of `scripts/lib/issueLicense.ts`), same form, but reachable from anywhere and able to **email the license straight to the customer** on submit instead of you doing it by hand. It's never linked from the public site nav, but a public URL protected by only a password still lets anyone on the internet *attempt* logins forever — see `ADMIN_ALLOWED_IPS` below. **Currently optional, deferred on purpose** (revisit before this handles real customer volume) — when unset, the portal falls back to password + rate-limiting alone, with no network-level gate in front of it.

One-time setup, in the `website/` Vercel project's env vars (Project Settings → Environment Variables):

Var	What it's for
<code>ADMIN_ALLOWED_IPS</code>	Optional (for now). Comma-separated IPv4 addresses/CIDRs allowed to reach any <code>/api/admin/*</code> route at all (e.g. <code>203.0.113.9</code> , <code>198.51.100.0/24</code> — your office/home IP, or a VPN egress range). Checked first, at the network level, independent of the password — see <code>website/lib/adminAuth.ts</code> 's <code>requireAllowedIp</code> . Unset skips this gate entirely — the portal still requires the password and is still rate-limited, it just no longer blocks unrecognized IPs before that. Set this once real customer volume makes the portal worth hardening further. Note this only gates the API calls (login, session check, issuing, history) — the static login page itself always loads for anyone who finds the URL either way.
<code>LICENSE_PRIVATE_KEY_PEM</code>	The exact contents of <code>secrets/license-private-key.pem</code> — this is what actually signs licenses, so it must be the same keypair the exe verifies against (<code>src/domain/licensePublicKey.ts</code>). Treat it with the same care as the file itself: paste it into Vercel's env var UI, never commit it.
<code>ADMIN_PASSWORD_HASH</code>	Generate with <code>bun run scripts/hash-admin-password.ts "a password"</code> — the plaintext password is never stored, only this hash.
<code>ADMIN_SESSION_SECRET</code>	Any long random string (e.g. <code>openssl rand -hex 32</code>) — signs the login session cookie.
<code>KV_REST_API_URL</code> , <code>KV_REST_API_TOKEN</code>	Optional. A Vercel KV (or any Upstash Redis) store's REST credentials. Without these, issuing + emailing still works, you just don't get a "previously issued" history list or login rate-limiting.
<code>AZURE_TENANT_ID</code> , <code>AZURE_CLIENT_ID</code> ,	Same Microsoft Graph credentials the contact form uses (see 9.6) — the portal reuses that mailbox to send license emails.

Var	What it's for
AZURE_CLIENT_SECRET , MAIL_SENDER_UPN	
CRON_SECRET	Any long random string (e.g. <code>openssl rand -hex 32</code>) — authorizes the daily expiry-reminder cron below. Vercel sends it automatically as a Bearer token when triggering the scheduled run; set the same value in the project's env vars.

Once configured: go to `https://<your-site>/admin`, sign in, fill in customer/plan/devices/expiry, and submit. The license is issued, emailed (with the `.license.key` file attached and pasted inline in the email body), and — if KV is configured — recorded so you can see it again later. A trash-can button on each history row removes it from this list — since licenses are verified entirely offline (no phone-home from the installed product), this is bookkeeping only and can't revoke a license file already delivered to a customer.

Automatic expiry reminders: `api/cron/expiry-reminders.ts` runs daily (`vercel.json` 's `crons`, 06:00 UTC) and emails any customer in the KV history whose license expires within 7 days, once each (tracked via a `expiryReminderSentAt` flag on the record so it doesn't repeat daily). Needs `CRON_SECRET` set (see table above) and the same KV + Azure/mail vars the portal already uses — if KV isn't configured there's no history to scan, so it's a no-op. Vercel Cron on the Hobby plan only fires jobs once a day, which this schedule already respects.

9.6 Website deploy details (Vercel, contact-form email)

`website/` (React + Vite + Tailwind + Framer Motion) is the only public-facing part of the project besides the product itself — the marketing/download site and the `/admin` license portal both live here, deployed as one Vercel project.

Dev:

```
cd website
bun install
bun run dev
```

Deploy:

1. New Vercel project, root directory `website/`.
2. Set env vars (Project Settings → Environment Variables) from `website/.env.example`: `VITE_CONTACT_EMAIL` / `VITE_COMPANY_NAME` / `VITE_COMPANY_URL` (what's shown on the site itself), plus the Azure/license-portal vars below. `VITE_DOWNLOAD_URL` can be left unset — it defaults to the same-origin path described in §9.1. The site deliberately shows no version/size text, so nothing here needs updating release to release.
3. `api/contact.ts` and everything under `api/admin/` are picked up automatically as Vercel serverless functions — no extra config.

Contact form → Microsoft Graph setup (needs a Microsoft 365 / Azure AD admin, one-time): the contact form sends mail via Graph app-only auth (client-credentials), never a mailbox password.

1. **Azure Portal → Microsoft Entra ID → App registrations → New registration.** Any name (e.g. "Argus Website Contact Form"). Single tenant is fine. No redirect URI needed — this is a daemon/app-only flow, not interactive login.
2. Note the **Application (client) ID** and **Directory (tenant) ID** from the app's Overview page.
3. **Certificates & secrets → New client secret.** Copy the secret **value** immediately (shown once) → `AZURE_CLIENT_SECRET`.
4. **API permissions → Add a permission → Microsoft Graph → Application permissions → Mail.Send**. Then **Grant admin consent** for the tenant — application permissions don't work without this, and only a tenant admin can click it.
5. Pick (or create) a real mailbox the app will send **as** — e.g. `noreply@yourdomain.com`. It needs an actual Microsoft 365 mailbox license (Graph `sendMail` sends through a real mailbox, there's no anonymous relay). Set its address as `MAIL_SENDER_UPN`.

6. Set `CONTACT_TO_EMAIL` to where you want the form to land (should match `VITE_CONTACT_EMAIL`).

Without all of the above configured, submitting the form returns a clear error telling the visitor to email you directly — it never fails silently or claims success without actually sending.

Updating for a new release: `bun run release` in the repo root already copies the new build into `website/public/downloads/` under fixed filenames. Commit those files and redeploy — the Download button always serves whatever's there, no env vars to update for this.

9.7 Setting up the escalation hierarchy (Tier 1 / 2 / 3 contacts)

"Tier 1 / Tier 2 / Tier 3" is a **device group's escalation chain**: if an alert on a device in that group goes unacknowledged, Argus notifies the device's owner immediately, then walks down the chain — Tier 1 after however many minutes you set, Tier 2 after that, and so on — stopping the moment anyone acknowledges. To set it up:

1. **Invite each tier contact as a user first**, with their real name reflected in their email and a role — Admin → Users → "Invite a user" (email, role, temporary password they change on first login). A tier contact needs an account to be assignable and to be able to acknowledge an alert from the email/webhook it triggers.
2. **Go to Inventory**, scroll to the "Groups & escalation hierarchy" panel at the bottom, and click "Edit escalation" on the relevant group (or "Add group" first if the group doesn't exist yet).
3. Click "Add tier" for each level you want — pick the contact from the dropdown (the users you invited in step 1) and set "After (minutes)": how long an alert stays unacknowledged before that tier is notified. Tiers fire in the order listed, each one's delay counted from when the alert opened, not from the previous tier.
4. Save. Assign devices to this group from the Inventory device editor's "Group" field — any device without a group has no escalation chain (still alerts its individually-assigned owner, if any, but nothing beyond that).

Each tier contact needs SMTP and/or webhook notifications configured for themselves at Settings → Notifications (or instance-wide SMTP configured by an admin) to actually receive anything — the escalation chain decides *when* someone is notified, not *how*.

10. Client setup walkthrough

Hand this section to a customer as-is, or point them at your hosted download page, which covers the same ground.

System requirements: Windows 10 or 11, 64-bit, administrator rights to install (needed once, to register the background service).

1. Download `Argus-Setup-vX.Y.Z-win-x64.exe` (and, optionally, its `.sha256`).
2. *(Optional, recommended)* Verify the checksum in PowerShell:

```
Get-FileHash .\Argus-Setup-vX.Y.Z-win-x64.exe -Algorithm SHA256
```

and compare against the download page.

3. Run it. A normal Windows installer wizard opens — pick an install folder (or accept the default), optionally tick "Create a desktop shortcut," and finish. No console window at any point. Argus installs itself as a background Windows service (starts now, and automatically on every future boot — closing anything or logging out never stops monitoring) and offers to open the dashboard in your browser.
4. The dashboard opens at `http://localhost:58070` . Create the admin account (email + password, 10+ characters) and accept the terms.
5. **Discovery** → enter your subnet (e.g. `192.168.1.0/24`) → Start scan → review results → Add devices. Monitoring more than one site or subnet (VPN, multiple offices)? See §5 first.
6. **Settings** → **Notifications** → configure SMTP and/or a webhook, send a test to confirm delivery, then set your own per-user preferences further down the same page.

7. (If you have more than one person who should be notified when something breaks) **Admin** → **Users** → invite Tier 1/2/3 contacts by name and email, then **Inventory** → set up each group's escalation chain — see §9.7.
8. **Settings** → **License** → paste the `.license.key` file's contents you received by email → Apply license.
9. To reopen the dashboard later, use the "Argus Dashboard" shortcut the installer created (Start Menu or desktop) — it's already running in the background, so this just opens a browser tab, no relaunch needed. To fully remove Argus, use "Uninstall Argus" from the Start Menu or Windows Settings → Apps — this stops and removes the service but leaves your device data behind in case you reinstall later.

11. Verification checklist

Run through this once after any new build, and once before handing a copy to a real customer.

- ☐ Running the installer shows a real wizard (no console window at any point) and finishes with the dashboard opening at `localhost:58070`
- ☐ After install, the Windows service (Services → "Argus Monitoring") is running, and a fresh reboot leaves it running without anyone logging in
- ☐ The "Argus Dashboard" Start Menu / desktop shortcut opens the browser without ever flashing a console window
- ☐ Setup completes and creates the admin account
- ☐ Sign out / sign back in works
- ☐ Discovery finds real devices on a test subnet
- ☐ Importing discovered devices adds them to Inventory with correct default checks
- ☐ Dashboard shows a green "Live" indicator and updates without a manual refresh
- ☐ Test email actually arrives; test webhook (if configured) actually arrives
- ☐ Adding a device with an unreachable IP produces an alert within one poll cycle, and the configured notification fires
- ☐ Acknowledge / Resolve on an alert works
- ☐ Inventory → a group's "Edit escalation" saves a multi-tier chain, and each tier fires (email arrives) only after its configured delay on an unacknowledged alert
- ☐ Map renders; zoom (scroll), pan (drag background), and the on-screen controls all work
- ☐ Reports shows data and the CSV export downloads
- ☐ Download backup produces a file; Restore from that file succeeds and the app restarts
- ☐ Stopping and restarting the "Argus Monitoring" service (or rebooting): all devices/alerts/ settings persisted
- ☐ License: trial mode shows a 5-device cap; applying a real `.license.key` shows the correct customer/plan/expiry and lifts the cap; a device beyond the cap is correctly refused
- ☐ `bun run license-admin` opens, issues a license identical in effect to the CLI, and lists it afterward
- ☐ Uninstalling removes the service and program files but leaves the `data` folder behind
- ☐ The downloaded installer/exe is reachable and checksums with **no GitHub login prompt**, from a signed-out browser session or `curl` with no auth
- ☐ (Before enabling for customers) Check for updates → Update now completes end-to-end on a real machine, not just in development, using a feed signed by `scripts/sign-release.ts`
- ☐ (Multi-site deployments) From the Argus machine, `ping` / `Test-NetConnection` each remote subnet's gateway to confirm the VPN actually routes there before assuming a remote device's "down" alert is real

12. Troubleshooting

Symptom	Fix
Port 58070 already in use	Argus refuses to start and logs <code>port_in_use</code> — stop whatever else is bound to it, or set <code>PORT</code> / <code>config.json</code> 's <code>port</code> to something else. It deliberately never drifts to a different port silently (see <code>data/logs/argus.log</code>)
Other machines on the LAN can't be discovered/reached	Run <code>Argus.exe --fix-firewall</code> (from an elevated prompt, in the install folder) and apply the printed commands
A remote-site device always shows down, but you can RDP/ping it from elsewhere	The Argus <i>host</i> likely can't reach it — see §5: check VPN routing, overlapping subnets, and firewall rules between the Argus machine and that site specifically
Dashboard shortcut doesn't load anything	The service may still be starting, or isn't running — check Services → "Argus Monitoring" is "Running"; if the service failed to register, re-run the installer or <code>Argus.exe --install-service</code> as Administrator
A device's IP changed	IPs are immutable after creation — delete and re-add
Test email fails	Check host/port/TLS — most providers want port 587 with TLS, or 465 with implicit TLS
License won't apply	Make sure the entire <code>.license.key</code> contents were pasted, including both parts either side of the <code>.</code>
"Update check URL must be https://"	Enforced deliberately — see §6; the feed and download must both be served over TLS
Self-update says "SERVICE_INSTALLED"	Installed via the installer or <code>--install-service</code> ? Stop the service, replace the exe by hand, start it again — auto-update deliberately refuses to race the service manager
Downloader gets a GitHub login prompt	Your download link points at a <i>private</i> repo's Releases — see §9.1; move the binaries to a public releases-only repo or R2, never make the source repo itself public to fix this
A group's escalation tier never fires	Confirm the assigned user has a notification channel configured (Settings → Notifications, per-user section) and isn't disabled (Admin → Users)
Installer requires admin / triggers UAC	Expected — registering a Windows service requires it. The portable exe (secondary download link) needs no elevation but also won't install a service

13. Reference: CLI flags, environment variables, file locations

CLI flags (`Argus.exe --help`) — mostly for troubleshooting or building from source; the installer already runs `--install-service` for you:

Flag	Effect
<code>--tray</code>	Run with a system-tray icon instead of a console window (still shows a console — use the installer's background service for a truly window-free run)
<code>--install-service</code>	Register as a Windows service (survives reboots) — what the installer does automatically
<code>--uninstall-service</code>	Remove that service — what the installer's uninstaller does automatically
<code>--fix-firewall</code>	Print (never run) the <code>netsh</code> commands to open the firewall

Flag	Effect
<code>--version</code> , <code>-v</code>	Print the version and exit
<code>--help</code> , <code>-h</code>	Show help and exit

Environment variables (override `config.json` , which is created on first run):

Variable	Meaning
<code>PORT</code>	default <code>58070</code> — fixed, fails fast (doesn't auto-increment) if already in use
<code>DATA_DIR</code>	default <code>./data</code> . The installer wizard prompts for this on a fresh install ("Choose Data Location", right after picking the install folder) and writes the chosen path into <code>config.json</code> — pick a different drive here if you want monitoring history kept off the OS disk. Only asked once: reinstalling over an existing install (i.e. <code>config.json</code> already exists) skips the prompt and keeps whatever's already configured.
<code>LOG_LEVEL</code>	<code>debug</code> / <code>info</code> (default) / <code>warn</code> / <code>error</code>
<code>INSTANCE_NAME</code>	default <code>Argus</code>
<code>UPDATE_CHECK_URL</code>	must be <code>https://</code> ; empty disables auto-update

Remote agent only (read by `bun run agent` / `src/bootstrap/agentMain.ts` — not by the main exe; see §5.1):

Variable	Meaning
<code>ARGUS_AGENT_TOKEN</code>	Required. The one-time token shown when the agent was created (Admin → Remote Agents)
<code>ARGUS_CENTRAL_URL</code>	Required. The central Argus instance's base URL the agent reports back to
<code>ARGUS_AGENT_INTERVAL_MS</code>	How often the agent polls its assigned devices, in milliseconds (default <code>30000</code>)

Build-time only (read by `scripts/release.ts` / `scripts/license-admin.ts` , not by the shipped exe):

Variable	Meaning
<code>ISCC_PATH</code>	Explicit path to Inno Setup's <code>ISCC.exe</code> , if it's not in one of the default install locations <code>bun run release</code> already checks
<code>COMPANY_NAME</code>	Stamped onto the exe's version info and the installer's publisher field (defaults to "Argus")
<code>PORT</code> (for <code>license-admin</code>)	Port the license-issuer UI binds to on <code>127.0.0.1</code> (default <code>4790</code>)

Where things live (relative to the exe, inside `DATA_DIR` — same folder the installer put `Argus.exe` in, by default `Program Files\Argus`):

Path	Contents
<code>data/argus.db</code> (+ <code>-shm</code> / <code>-wal</code>)	The SQLite database — devices, metrics, alerts, users, settings
<code>data/instance.key</code>	Random per-install key deriving encryption for stored secrets
<code>data/logs/argus.log</code>	Application log
<code>data/license.key</code>	The currently-applied license file, if any
<code>data/argus.lock</code>	Single-instance lock (prevents two copies running against the same data)

Path	Contents
<code>secrets/license-private-key.pem</code>	Your license-signing key — never ships, never commit
<code>secrets/release-private-key.pem</code>	Your update-feed-signing key — never ships, never commit
<code>tools/Argus-service.exe</code>	Your own copy of WinSW, referenced by the installer build (not committed — see §8)
<code>issued-licenses/</code>	Every <code>.license.key</code> <code>scripts/license-admin.ts</code> has issued (gitignored)
<code>installer/argus.iss</code>	The Inno Setup source script that builds <code>Argus-Setup-*.exe</code>
<code>config.json</code>	Created on first run from <code>config.example.json</code> ; edited via Settings → General or by hand